Experiment Response Monitor - Project Report

Oct 8, 2026 (scoping) · Updated Oct 22, 2026 (as-built) · @bhavika · Repository: github.com/bhavika-chandru/experiment-response-monitor

"Experiment Response Monitor" is my Level 3 (advanced) project for the no-code/n8n system design course: a workflow that watches live experiment responses as they arrive and flags the ones that need a person to look at them, instead of relying on someone watching the dashboard. This document keeps the original scoping (sign-off before building) and adds a final section describing exactly what was built, run, and tested.

The problem

During a live session (the kind of Qualtrics study the lab runs, e.g. VRAC), responses arrive one at a time while the session is still open. Catching a bad one today means a research assistant watching the dashboard by eye — a skipped attention check, a duplicate submission, a response filed under the wrong condition. Nothing alerts anyone in real time; problems usually surface during cleanup, days after the session closes.

Four questions set the scope:

- **What happens in the real world:** a participant submits a response in Qualtrics; it should be checked against a few rules the moment it lands, and the team should hear about anything off immediately, not at cleanup.
- **What data exists, and where:** the response — participant ID, session ID, timestamp, embedded condition data, attention-check answer, completion duration — lives in Qualtrics. This project reads that data; it doesn't own it.
- **What triggers what:** a completed response in Qualtrics (polled export) is the primary trigger; a manual/Slack-initiated check is also supported for on-demand testing. No human has to click through a dashboard.
- **What's allowed to break:** the monitor, never the survey. If the monitor goes down, participants keep completing the study with zero disruption — it just means the team catches problems later instead of immediately, which is where things stood before this project.

Architecture

Qualtrics is the external service and the database of record for raw responses; n8n is the backend; Postgres is the durable system-of-record log (added during build — see Implementation Update); the Google Sheet is a lightweight human-readable cache used for the duplicate check; Slack is the alerting channel, reached on every flagged response and separately for monitor failures.

Validation rules

Every response runs through one Switch node, in order. The first rule it matches decides the flag; a response matching none of them is clean.

Rule	Condition	Flag
Incomplete	A required field (participant ID or condition) is missing	incomplete
Attention check	The attention-check item was answered wrong	attention_check_fail
Duration outlier	Completion time is below/above the expected range (60–1800s)	duration_outlier
Branch mismatch	Recorded condition doesn't match the embedded assignment	branch_mismatch
Duplicate	Response ID already logged (checked against the Sheet before entering the Switch)	— filtered before reaching Switch
None of the above	—	clean, logged, no alert

This is the Level 3 part of the scope: five branches on one Switch node, not a single If, each one a real rule from running these sessions rather than a toy example.

Scope as originally proposed

- **Won't touch the live survey.** The monitor only reads from Qualtrics; it never modifies, pauses, or gates a participant's session.
- **Qualtrics stays the system of record.** The log sheet / database is a cache built to catch duplicates and keep a durable trail — not a second copy intended to replace Qualtrics.
- **One study at a time.** Built against the current Qualtrics survey structure; adapting it to a second study is follow-on work.

Implementation Update — as built and demoed (Oct 22, 2026)

The system described above was built, self-hosted via Docker Compose, and tested end-to-end against real Qualtrics data and a live Slack workspace. One deliberate scope change from the original proposal: an AI node was added (originally deferred as "No AI in v1") to generate a natural-language explanation of each flag, posted as a second Slack message alongside the deterministic rule-based alert — the rule-based decision logic itself remains fully deterministic; the AI only explains it in plain language.

Minimum technical requirements — coverage

Requirement	Where it's used
If or Set node	Edit Fields (Set) nodes build each flag's payload; If nodes (If2, If1) gate export-status polling and duration formatting.
Code node	A JavaScript Code node runs the duplicate-response filter against the logged response IDs.
Router / If / Switch statement	A single Switch node with 5 ordered routing rules (incomplete → attention_check_fail → duration_outlier → branch_mismatch → clean).
API or MCP connection	Qualtrics REST API (export job + download), Slack API, Google Sheets API, and a PostgreSQL connection.
Message trigger (Slack, not Chat Message)	A Slack Trigger node feeds into the same export chain as the scheduled trigger, so the monitor can also be kicked off from a Slack event.
Database for data persistence	Self-hosted PostgreSQL (Docker container) logs every response (id, response_id, participant_id, timestamp, flag, note) as the durable system of record.
AI node (LLM Chain / AI Agent)	A Basic LLM Chain node, backed by Google Gemini, generates a one-sentence natural-language explanation of each flag, posted to Slack.
Sub-workflow	See note below.
Error handling and retries	An Error Trigger workflow is attached via Settings → Error Workflow and posts a separate Slack alert on any unhandled failure; Retry On Fail is enabled on the external API call nodes (Qualtrics export/download, Slack, Postgres).

Sub-workflow note: the error-handling workflow (Error Trigger → Slack) is maintained as a separate workflow file and attached at the workflow-settings level for automatic error alerts. On top of that, an explicit Execute Workflow node ("Call 'Experiment Monitor - Error Handler'") was added to the main workflow, wired from "Append row in sheet"'s success output alongside the existing Postgres/Sheets/AI branches — so the sub-workflow is now invoked as a normal step on every logged response, not only on failure. The called workflow was given a second trigger ("When Executed by Another Workflow") so it can be invoked this way in addition to its original Error Trigger. Verified working end-to-end (see Appendix Figure 1).

What was actually tested

- Self-hosted via docker-compose (Postgres + n8n containers), confirmed healthy and running independently of n8n Cloud.
- Real Qualtrics export pulled, parsed, and routed through the Switch — verified live against five freshly submitted responses, landing correctly in three branches: clean, attention_check_fail, and duration_outlier (see Postgres query, Appendix Figure 4).
- Duplicate-response filtering verified live: submitting the same response twice correctly produced only one logged row, with the repeat silently filtered before reaching the Switch.
- Every exercised branch logs to Postgres and the Google Sheet; a flagged response also triggers a Slack alert with the correct reason text and a separate AI-generated summary.
- Execute Workflow node verified live: the main workflow now calls the error-handling workflow directly as a normal step on every logged response (not only on failure), confirmed with a green run end-to-end.
- Error Workflow linked natively (Settings → Error Workflow) so any unhandled node failure triggers the error-handler automatically, posting workflow name and execution URL to Slack; Retry On Fail set on every external call (Qualtrics HTTP chain, Postgres, Gemini); AI output passed through an If-node schema check with a fallback message if empty.
- Known gap: branch_mismatch is not yet exercisable with live data — the "Map Qualtrics Fields" node currently hardcodes both condition and embedded_condition to the literal string "demo" rather than pulling them from the response, so the two can never disagree. The Switch branch and its downstream logging/alerting are implemented and were validated with the mock trigger; wiring both fields to real expressions is the fix, not yet done.
- Git repository with real commit history: github.com/bhavika-chandru/experiment-response-monitor

Failure handling

Failure	Setting	Behaviour
Qualtrics or Slack call times out (5xx/429)	Retry On Fail	Retries automatically; never retries a 4xx
A node in the chain dies	On Error: continue using error output	Routes to the error workflow instead of killing the run
Nobody notices it's broken	Error Trigger → error workflow	A separate Slack alert, so a dead monitor is itself visible
Slack alert itself fails to send	Retry, then log	Logged to Postgres/Sheet rather than silently dropped

What I'd cut first if short on time (as scoped originally): the AI-generated explanation layer and the dual Postgres+Sheets logging would be the first things simplified back to a single log destination and rule-based-only alerts — the five-branch Switch and the duplicate check are the non-negotiable core.

Appendix — full workflow canvas

The complete n8n workflow, as self-hosted and demoed (green checkmarks: a full successful test run). The top path (Edit Fields → Normalized Input → Get row(s) in sheet → Code in JavaScript → Switch) is the duplicate filter and the five-branch routing logic; each branch's Edit Fields node feeds into Execute a SQL query (Postgres, Retry On Fail enabled) and Append row in sheet (the Sheets cache), which in turn feeds the Google Gemini / Basic LLM Chain (Retry On Fail enabled) into an If node that checks the AI output and falls back to a default message if it's empty — both paths converge on the same Slack node — the flagged-alert Slack path, and the Execute Workflow node calling the error-handler workflow directly as a normal step. The error-handler workflow is also linked as this workflow's native Error Workflow (Settings → Error Workflow), so any unhandled node failure triggers it automatically, posting the workflow name and execution URL to Slack. The bottom path (Schedule Trigger / Slack Trigger → Start Export → Check Export Status → Download Export → Parse Responses → Map Qualtrics Fields) is the Qualtrics export-and-poll chain feeding into the top path, with Retry On Fail set on every HTTP call.

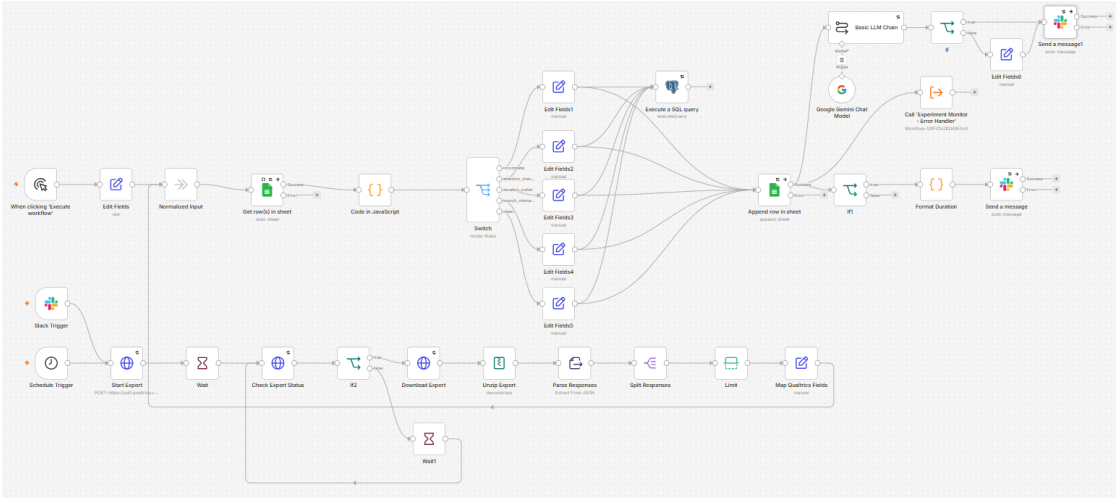


Figure 1: n8n workflow canvas — full pipeline from trigger to Slack alert, including the Error Workflow link and AI output fallback path.

Self-hosting, confirmed running. Output of `docker compose ps` against the live stack: the n8n container and the Postgres container both up for the full working session, Postgres reporting healthy.

```
PS C:\Users\Bhavika\Downloads\erm-repo-clone> cd C:\Users\Bhavika\Downloads\erm-repo-initial
PS C:\Users\Bhavika\Downloads\erm-repo-initial> docker compose ps
WARN[0000] C:\Users\Bhavika\Downloads\erm-repo-initial\docker-compose.yml: the attribute `version` is obsolete, it will
be ignored, please remove it to avoid potential confusion
NAME                          IMAGE              COMMAND                  SERVICE    CREATED       STATUS
  PORTS
erm-repo-initial-n8n-1        n8nio/n8n:latest   "tini -- /docker-ent…"   n8n        3 hours ago   Up 3 hours
  0.0.0.0:5678->5678/tcp, [::]:5678->5678/tcp
erm-repo-initial-postgres-1   postgres:16.4      "docker-entrypoint.s…"   postgres   3 hours ago   Up 3 hours (healthy)
  5432/tcp
```

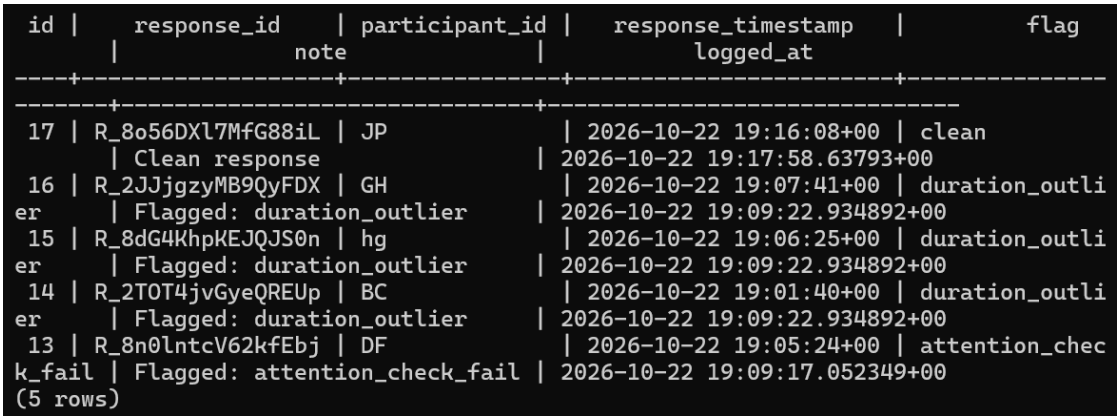
Figure 2: docker compose ps — n8n and Postgres containers both up, Postgres healthy.

Git history. Real, incremental commit history on the repository's main branch, pushed to GitHub (linked above).

```
* f258f44 (HEAD -> main, origin/main, origin/HEAD) Add Execute Workflow node
  calling error-handler as a normal step
* 2708ee9 Replace proposal PDF with as-built version including appendix
* d84caf1 Update README for self-hosted demo; replace proposal PDF with
  as-built version
* 7038364 Add working n8n workflows: main demo workflow with Postgres
  logging, Slack alerts, Gemini AI summary node, and error handler
* 8740157 Initial scaffold + proposal
```

Figure 3: git log --oneline --graph --decorate — commit history on main.

Database persistence, confirmed. Query against the live self-hosted Postgres instance, run straight from the "Execute a SQL query" node's own table: five real Qualtrics responses, submitted live and pulled through the full export-and-poll chain, landing in three different branches — clean, duration_outlier, and attention_check_fail — each with a correctly captured participant ID.



```
 id |    response_id    | participant_id |   response_timestamp   |         flag
      |             note              |          logged_at
----+-------------------+----------------+------------------------+--------------
-------+-------------------------------+-------------------------------
 17 | R_8o56DXl7MfG88iL | JP             | 2026-10-22 19:16:08+00 | clean
      | Clean response                | 2026-10-22 19:17:58.63793+00
 16 | R_2JJjgzyMB9QyFDX | GH             | 2026-10-22 19:07:41+00 | duration_outli
er     | Flagged: duration_outlier     | 2026-10-22 19:09:22.934892+00
 15 | R_8dG4KhpKEJQJS0n | hg             | 2026-10-22 19:06:25+00 | duration_outli
er     | Flagged: duration_outlier     | 2026-10-22 19:09:22.934892+00
 14 | R_2TOT4jvGyeQREUp | BC             | 2026-10-22 19:01:40+00 | duration_outli
er     | Flagged: duration_outlier     | 2026-10-22 19:09:22.934892+00
 13 | R_8n0lntcV62kfEbj | DF             | 2026-10-22 19:05:24+00 | attention_chec
k_fail | Flagged: attention_check_fail | 2026-10-22 19:09:17.052349+00
(5 rows)
```

Figure 4: Postgres query — live Qualtrics responses logged across three flag branches.